

# ORDBs

## Data models in database systems

### Lecture 7

Mara Romanovska

Mara.Romanovska@rtu.lv

09.10.2025.



**RTU**  
DATORZINĀTNES UN  
INFORMĀCIJAS  
TEHNOLOĢIJAS FAKULTĀTE

# Agenda

- Object methods
- ORDB design

# **Practical work 1 – now available in ORTUS**



# To do ASAP



URL

Registration for PW1 

To fill out the form, user must log in with his/her RTU credentials.

**Please note that each student must be registered in a group to gain access to PW1.**

Registration deadline: **Friday, 03.10.2025, 18:00. If you register late, access to the practical work variation may be delayed by up to a week.**

Please see the lecture slides for more information!

# Principles that apply

- Be sure to read all the materials
- Many things are optimised for more efficient work (including report)
  - Must submit PDF document
  - Link to LiveSQL must be provided
- Focus on quality, not quantity

# Specifics of LiveSQL

- Using TREAT() and IS OF – please be sure to check the sample script

# Object methods



# Methods

- Methods are functions or procedures
- They are declared in the type definition
- Methods create behaviour that can be carried out with object
- Principal usage of method is to access object's attributes
- To execute some operation the application calls the corresponding method from the corresponding object



# Types of methods

- Methods can be written in different languages
- Methods written in PL/SQL are stored in the database. Methods written in other languages are stored outside the database
- Main types of methods are:
  - *Member* methods
  - *Static* methods
  - *Map* methods
  - *Order* methods
  - Constructor methods

# Object comparison methods

- Scalar types have predefined ordering that is used to compare objects
- Such an order does not exist for objects that have more than one attribute
- To be able to order objects, we need to specify comparison order
- Ordering can be defined by two specific methods given in the type definition
  - Map methods
  - Order methods
- Each object type can define only one map or order method

# Static methods

- Are executed with object types not instances
- Are used for operations that are global for the type and do not use a specific instance
- Do not have parameter SELF
- The idea is identical to the static methods in modern programming languages like Java

# Constructor methods

- Each object type has a system defined constructor method that
  - Creates new object
  - Assigns values to attributes
- Constructor function returns new object as its value
- Its name is the same as the name of the type
- Parameters are the same as attributes of the object type by name, type and order

# Defining object methods in PL/SQL



# Full type definition. Example

```
CREATE OR REPLACE TYPE Student AS OBJECT(  
    ID                NUMBER,  
    NAME              VARCHAR2(25),  
    SURNAME            VARCHAR2(35),  
    GENDER             CHAR(8),  
    PERS_ID            CHAR(11),  
    ADDRESS            VARCHAR(60),  
    PHONE              VARCHAR(12),  
    CLASS              NUMBER,  
    MAP MEMBER FUNCTION Sort_Order RETURN NUMBER,  
    MEMBER FUNCTION Avg_Grade (S_ID IN NUMBER) RETURN NUMBER,  
    MEMBER PROCEDURE Change_Surname (S_ID IN NUMBER, NEW_SURN IN  
        VARCHAR2),  
    MEMBER PROCEDURE Change_Address (S_ID IN NUMBER, NEW_ADDR IN  
        VARCHAR2),  
    MEMBER PROCEDURE Change_Phone (S_ID IN NUMBER, NEW_Phone IN  
        VARCHAR),  
    MEMBER PROCEDURE Change_Class (S_ID IN NUMBER, New_Class IN NUMBER));
```

# Creating type body

```
CREATE OR REPLACE TYPE BODY Schema.name_of_the_type IS or
AS
    MAP or ORDER MEMBER function_definition,
    STATIC or MEMBER function_definition,
    STATIC or MEMBER procedure_definition, ...
END;
```

# Member methods

- Member methods implement access to data of instances
- A member method is defined for any operation done with object data
- Member methods have built in parameter SELF that denotes the instance processed by the method



# Member method. Example (1/4)

- The method that calculates the average mark for a student

```
CREATE OR REPLACE TYPE Student AS OBJECT (  
    ID          NUMBER,  
    NAME        VARCHAR2 (25) ,  
    SURNAME     VARCHAR2 (35) ,  
    GENDER      CHAR (6) ,  
    GRADE_MATH  NUMBER (2) ,  
    GRADE_LATV  NUMBER (2) ,  
    GRADE_FHYS  NUMBER (2) ,  
    AVG_GRADE   REAL,  
    MEMBER PROCEDURE Average_Grade) ;
```

# Member method. Example(2/4)

- Type body contains Member method code that calculates the average grade:

```
CREATE OR REPLACE TYPE BODY Student AS
    MEMBER PROCEDURE Average_Grade IS
        BEGIN
            SELF.AVG_GRADE := (SELF.GRADE_MATH+
            SELF.GRADE_LATV+SELF.GRADE_PHYS)/3;
        END Average_Grade;
END;
```

# Member method. Example (3/4)

- It is possible to calculate average grade for one student in the following way

```
DECLARE
```

```
    variable1      Student;
```

```
BEGIN
```

```
    SELECT  VALUE(A) INTO variable1 FROM Students A WHERE A.ID = 1;
```

```
    variable1.Average_Grade() ;
```

```
    DELETE FROM Students A WHERE A.ID = 1;
```

```
    INSERT INTO Students VALUES (variable1);
```

```
END;
```

# Member method. Example (4/4)

- Results of the calculation:

```
SELECT VALUE(A) FROM Students A;
```

```
VALUE(A) (ID,NAME,SURNAME,GENDER,GRADE_MATH,GRADE_LATV,GRADE_PHYS, AVG_GRADE)
```

```
-----  
STUDENT(3, 'Zane', 'Sakne', 'female', 7, 6, 7, NULL)
```

```
STUDENT(2, 'Varis', 'Celms', 'male', 6, 7, 6, NULL)
```

```
STUDENT(1, 'Liene', 'Koks', 'female', 8, 9, 8, 8.3)
```

# Calling member methods

- Parameters in MEMBER functions by default are IN parameters, in member procedures – IN OUT
- Member methods are called with dot notation **object.method()**
  - The name of the object is given on the left side and the name of the method follows
  - All parameters are written in parenthesis

# Map method

- *Map method* compares objects based on one of its attributes
- Map method returns scalar value (number, date, text, etc.)
- This method can be used to order any number of objects
- Keyword MAP is used to define such a method
- Method is executed when the comparison **object1>object2** is done

# Example of the MAP method

- We will compare students by their average grade

```
CREATE OR REPLACE TYPE Student AS OBJECT(  
    ID                NUMBER,  
    NAME              VARCHAR2(25),  
    SURNAME           VARCHAR2(35),  
    GENDER            CHAR(6),  
    GRADE_MATH        NUMBER(2),  
    GRADE_LATV        NUMBER(2),  
    GRADE_FHYS        NUMBER(2),  
    MAP MEMBER FUNCTION Student_Evaluation RETURN REAL);
```

```
CREATE OR REPLACE TYPE BODY Student AS  
    MAP MEMBER FUNCTION Student_Evaluation RETURN REAL IS  
    BEGIN  
        RETURN (SELF.GRADE_MATH+SELF.GRADE_LATV+ SELF.GRADE_PHYS)/3  
    END Student_Evaluation;  
END;
```

# Using MAP method

```
SELECT VALUE(A) FROM Students A ORDER BY VALUE(A) ;  
VALUE(A) (ID, NAME, SURNAME, GENDER, GRADE_MATH, GRADE_LATV, GRADE_PHYS)  
-----  
STUDENT(2, 'Varis', 'Celms', 'male', 6, 7, 6)  
STUDENT(3, 'Zane', 'Sakne', 'female', 7, 6, 7)  
STUDENT(1, 'Liene', 'Koks', 'female', 8, 9, 8)
```



# Order methods

- **Order methods** or sort functions execute direct object to object comparison
- These methods determine whether the current object is smaller, equal or greater than the object it is compared to
- Order method is a function with one parameter – another object of the same type (it is used to compare to)
- Method should be written so that it returns a negative number, zero or a positive number as a result of comparison. For example it can return the following values:
  - -1, if the value of the object given as a parameter is greater than SELF
  - 1, if the value of the object given as a parameter is smaller than SELF
  - 0, if the value of the object given as a parameter is equal to SELF

# Defining the order method

- An order method is defined as follows:

```
ORDER MEMBER FUNCTION Sort_Function (  
    attribute IN Object_Type_Name)    RETURN NUMBER;
```

# Example of order method

- Comparison of clients by ID numbers

```
CREATE TYPE Client AS OBJECT(  
  id          number,  
  name        varchar2(30),  
  ORDER MEMBER FUNCTION Compare (c Client) RETURN integer  
);
```

```
CREATE TYPE BODY Client AS  
ORDER MEMBER FUNCTION Compare (c Client) RETURN integer IS  
BEGIN  
  IF SELF.id < c.id Then RETURN -1;  
  ELSEIF SELF.id > c.id Then RETURN 1;  
  ELSE RETURN 0;  
END IF;  
END Compare;  
END;
```

# Calling the order method

```
DECLARE
```

```
...
```

```
BEGIN
```

```
...
```

```
A := O1. compare(O2)
```

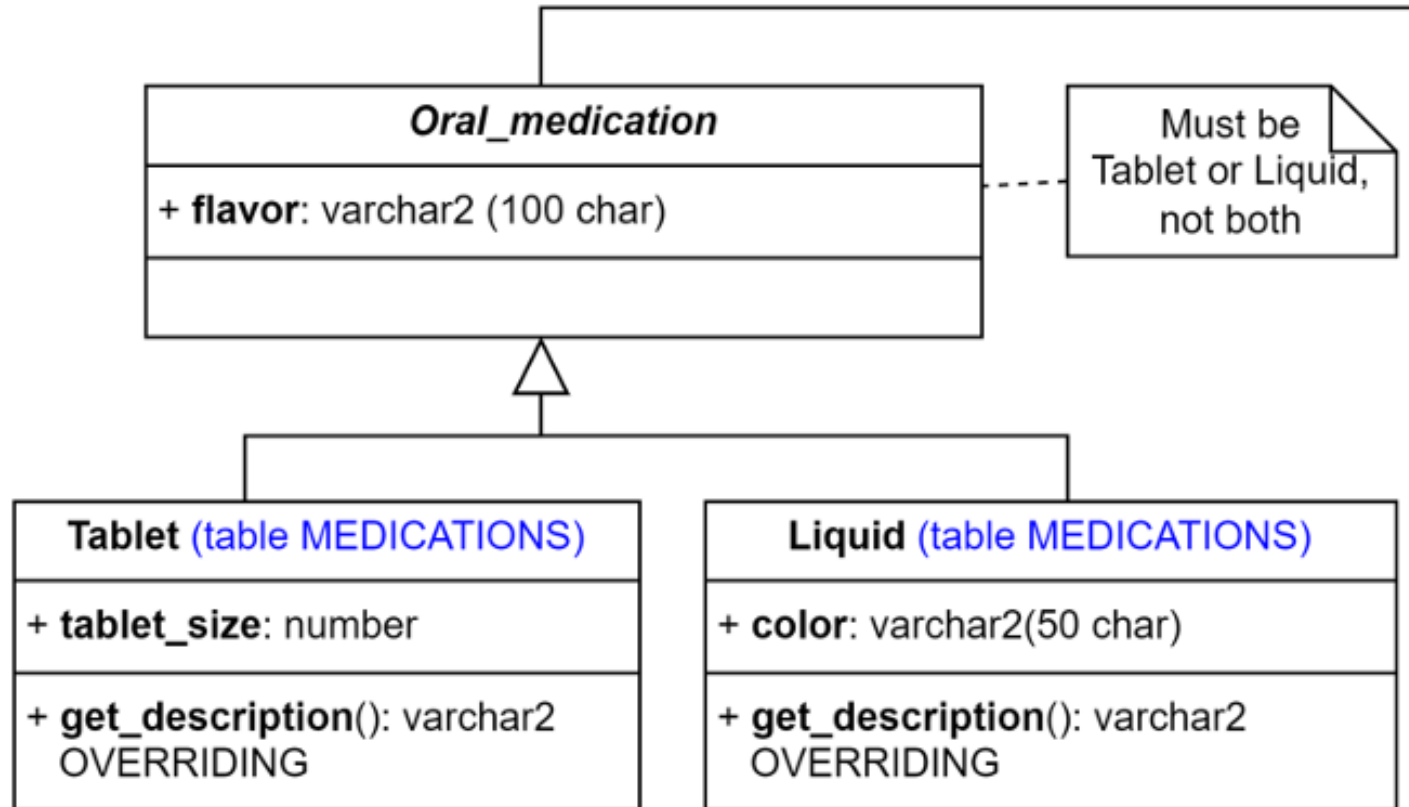
```
...
```

```
END
```

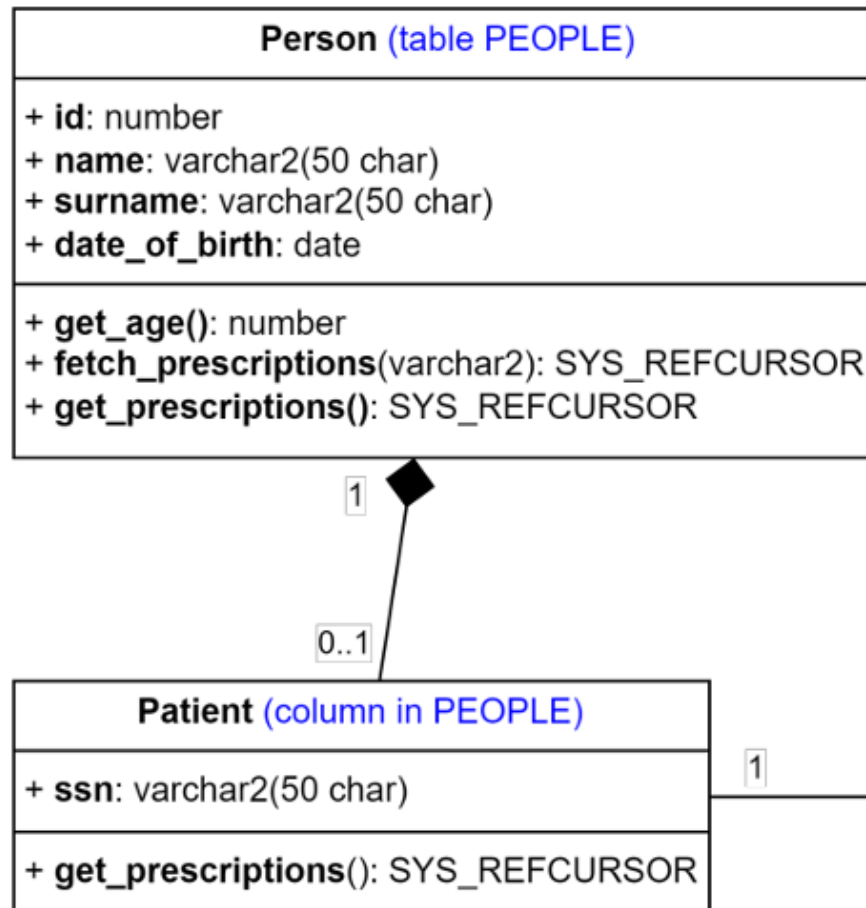
# ORDB design



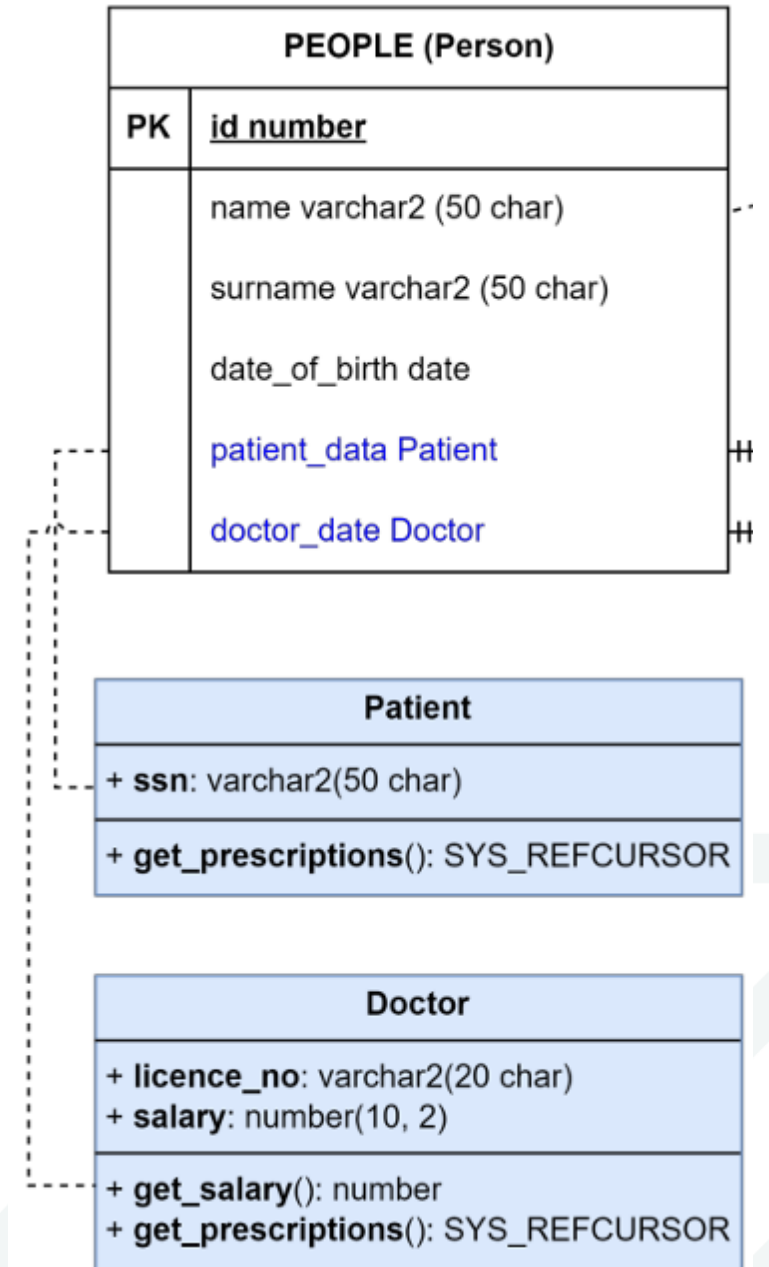
# Class diagram (1)



# Class diagram (2)



# Extended ER diagram





# Thank you!

